

Fuzzing guiado por IA para contratos Soroban

Uma ferramenta que testa um contrato inteligente com milhares de sequências de operações que ninguém escreveu — e sabe dizer quando algo saiu errado.

Os bugs mais caros não quebram nada

Um contrato pode aceitar um depósito de zero, trocar o administrador sem permissão, ou deixar um saldo parar de fechar — e continuar respondendo normalmente. Nenhum erro, nenhum alarme. Só o estado errado, esperando alguém explorar.

O teste comum não vê

Ele confere os casos que o desenvolvedor imaginou. O bug mora justamente no caso que ninguém imaginou.

A revisão humana não escala

Uma auditoria séria leva dias de uma pessoa cara, e um contrato muda a cada semana.

Soroban tem armadilhas próprias

Dados que expiram com o tempo, três camadas de armazenamento, autorização por endereço. As ferramentas de Ethereum não se aplicam.

Fuzzing: em vez de escrever testes, **sortear milhares deles**

COMO FUNCIONA

O fuzzer monta sequências aleatórias de operações — deposita, saca, transfere, avança o tempo, tenta sem permissão — com valores que uma pessoa não escolheria: zero, um, o maior número possível, exatamente o saldo mais um.

ONDE É FORTE

Explorar. Ele chega em combinações de estado que nenhum caso de teste escrito à mão alcança, e faz isso em minutos.

ONDE É CEGO

Ele só sabe que algo *quebrou*. Um saldo errado não quebra nada — e o fuzzer passa reto. Para ver isso ele precisa de **regras**: "o total de cotas é sempre a soma das cotas de cada um".

Escrever essas regras é a parte cara. Exige entender o contrato a fundo, conhecer as armadilhas da plataforma, e imaginar o que *poderia* dar errado. É trabalho de auditor sênior — e é exatamente o que uma IA que lê código faz bem.

A IDEIA

A IA escreve as regras. O fuzzer verifica.

Dois papéis, cada um no que faz melhor. A IA lê o contrato e propõe o que nunca pode acontecer. O fuzzer tenta fazer acontecer, milhares de vezes, e mostra a sequência exata quando consegue.

Um cofre com sete bugs plantados

Escrevemos um contrato de cofre em Soroban e plantamos sete defeitos reais, cada um escondido atrás de uma chave: aritmética que dá a volta, permissão que falta, depósito de zero, transferência para si mesmo, dados que expiram cedo, reinicialização indevida. A ferramenta nunca vê os bugs — só o contrato limpo.

A PERGUNTA

Ligamos um bug de cada vez. Quantos dos sete a ferramenta encontra?

A LINHA DE BASE

1/7

fuzzing sem regras — só vê o que quebra

OS CONTROLES

O contrato limpo tem que passar. Uma regra que cita o bug pelo nome invalida a rodada. Nenhuma conclusão sai de uma execução só.

RESULTADO

1 de 7 → 3 a 4 de 7 por execução, 6 de 7 em três

FUZZING SEM REGRAS

1/7

a linha de base

COM REGRAS ESCRITAS PELA IA

3–4/7

uma execução, sem nenhuma pessoa no meio

TRÊS EXECUÇÕES SOMADAS

6/7

cada rodada propõe regras diferentes

Cada execução custa **US\$ 0,25** e leva de 4 a 10 minutos. Três execuções somadas custam menos de um dólar e encontram seis dos sete defeitos — sem que ninguém tenha escrito um teste.

O que acontece quando você clica em Auditar

1 • LER

A IA lê o contrato inteiro e propõe as regras: o que nunca pode acontecer, com o que assume e quanto confia em cada uma.

2 • REVISAR

Uma segunda passada de IA descarta regras que não podem falhar ou que não dá para observar — o que sobra é o que vale testar.

3 • MONTAR O SIMULADOR

A ferramenta gera um ambiente com o contrato, usuários, dinheiro e tempo — e um sorteador de operações que prefere os valores perigosos.

4 • FUZZAR

Centenas de sequências sorteadas. Depois de **cada** operação, cada regra é conferida — comparando o antes com o depois.

5 • REPORTAR

O que falhou vem com a sequência mínima que quebra. O que passou vira uma suíte de testes pronta para o repositório.

Aponte para o contrato. Clique. **Leia o relatório.**

A INVESTIGAR

Regras que falharam, cada uma com a sequência exata de operações que a quebrou. Pode ser um bug do contrato, pode ser a regra errada — a ferramenta mostra e a pessoa decide.

VERIFICADAS

Regras que resistiram a todas as sequências. Ficam como testes de regressão: se alguém quebrar uma no futuro, o CI avisa.

NÃO VERIFICADAS

Regras propostas que precisam de mais contexto para virar teste, cada uma com o motivo. Ficam listadas como ponto de partida para quem for aprofundar.

0

alterações no seu repositório — tudo roda numa cópia

diff

de tudo que a IA escreveu, linha por linha

US\$

custo real de cada auditoria, no rodapé do relatório

A regra que pegou a aritmética errada

O QUE A IA PROPÔS

"As cotas que um depósito devolve têm que ser iguais a *valor × total de cotas ÷ ativos*, calculado com aritmética que detecta estouro. Se a minha conta estoura, o contrato tinha que ter recusado."

O QUE O FUZZER FEZ

Deu a cada usuário uma quantia enorme — grande o bastante para uma multiplicação passar do limite do número — e sorteou depósitos até um produto dar a volta. O contrato com o bug devolveu um número de cotas que não era o da conta.

O fuzzer só encontra o que consegue alcançar. Metade do valor desta ferramenta está no simulador: dinheiro suficiente para a conta estourar, tempo que avança até os dados expirarem, tentativas sem permissão — **as condições em que o defeito aparece.**

Auditoria contínua, a cada mudança

ANTES

Uma auditoria por versão, dias de trabalho de uma pessoa sênior, feita quando dá tempo — e o contrato muda toda semana.

AGORA

Uma auditoria por pull request. Minutos, centavos, sem esperar ninguém. O que passou vira teste permanente; o que falhou chega com a sequência exata para reproduzir.

PARA QUEM AUDITA

Começa com trinta regras propostas, um simulador pronto e os alarmes já reproduzidos — em vez de uma página em branco. O tempo caro vai para decidir, não para montar.

US\$ 0,25 minutos qualquer
por auditoria não dias contrato Soroban, sem adaptação

De protótipo a produto

- **Rodar no CI.** A ferramenta já roda sozinha e já entrega o custo; falta o gatilho por pull request e o comentário automático com os achados.
- **Mais execuções, mais cobertura.** Cada rodada propõe regras diferentes; somar três já encontra mais que qualquer uma. Fazer isso por padrão é uma linha de configuração.
- **Ampliar para os contratos reais da Stellar.** O simulador já monta e compila nos exemplos oficiais — token, timelock, atomic swap. O próximo passo é medir a qualidade das regras neles como medimos no cofre.
- **Uma biblioteca de regras por tipo de contrato.** As regras que a IA propõe para um cofre se repetem em todo cofre. Guardá-las e reaproveitá-las encurta cada auditoria seguinte.

Uma ferramenta, um método, um kit

A ferramenta

Web local, aponta para qualquer contrato Soroban, roda sozinha, relatório em três grupos, custo real, diff. Pronta para rodar a cada pull request.

O método

O que esconder do modelo, por que o contrato limpo roda primeiro, como separar achado real de alarme falso. Escrito para outra pessoa aplicar.

soroban-fuzzkit

As peças de fuzzing que não dependem de contrato nenhum: valores perigosos, pool de usuários, sondagem de tempo de vida. 40 testes.

EM UMA FRASE

O fuzzer sempre soube **explorar**. O que ele não sabia era o que procurar.

A IA escreve o que procurar. Juntos, encontram seis dos sete defeitos por menos de um dólar — sem que ninguém escreva um teste.

< stellar-studies >